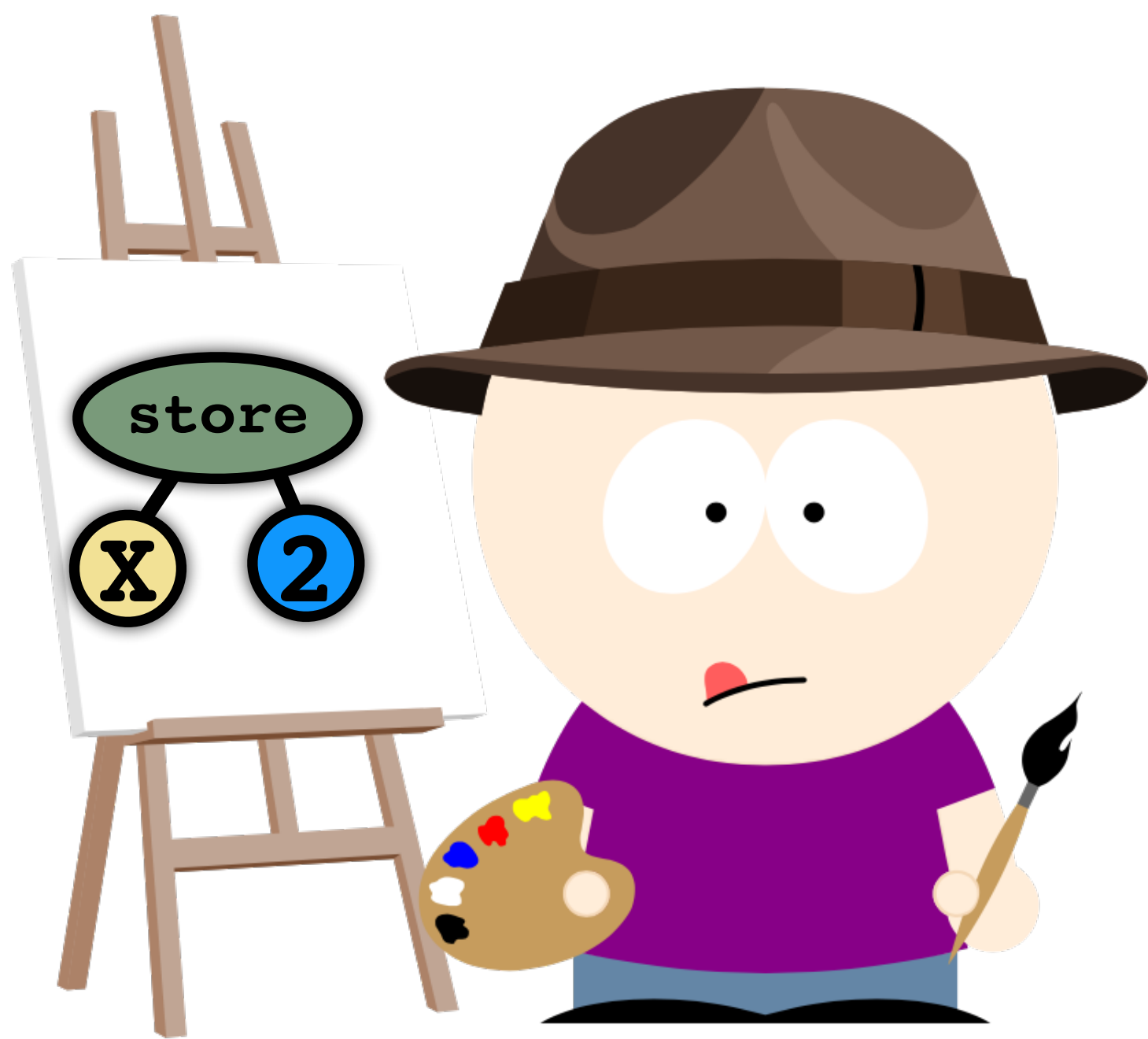




IN-CLASS EXERCISES



INTERMEDIATE CODE 1

Download exercises

1. Go to

<https://ligerlabs.org/compilers.html>

2. Download the file

`interm-1-exercises.zip`

3. Open up a terminal and Unzip the file

```
> unzip interm-1-exercises.zip
```

```
> cd interm-1-exercises
```

```
> ls
```

Task 1

- Translate this program into 3 kinds of IR:
 1. Sequence of expression trees
 2. Sequence of DAGs
 3. Quadruples

```
int main() {  
    int a = 2;  
    int b = a*3 + 2;  
    int c = a*3 + 5;  
    printf("%i", b+c);  
}
```


Task 2

- Translate this program into 3 kinds of IR:
 1. Sequence of expression trees
 2. Quadruples

```
int main() {  
    int a = 2;  
    int b[5];  
    b[a] = a*3;  
    int x = b[2*a] + 42;  
}
```

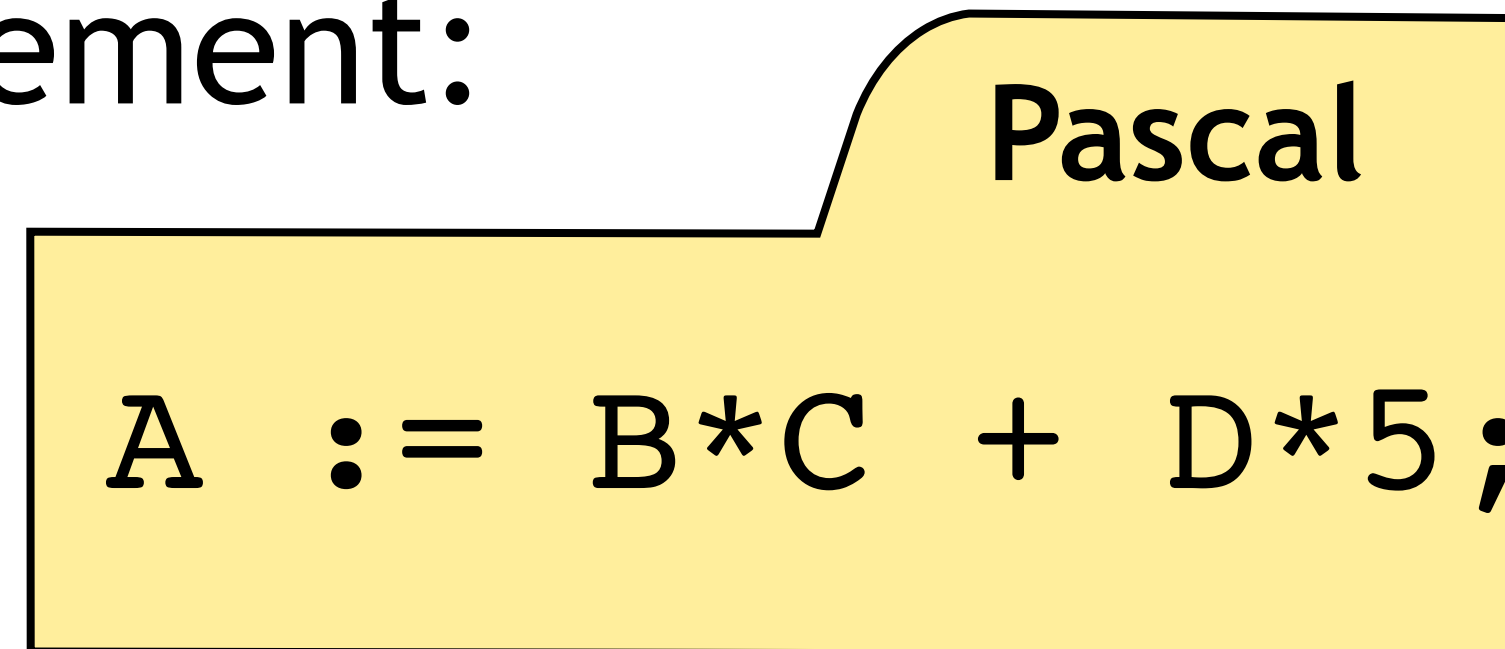
Task 3

- Translate this program into quadruples:

```
void foo(int n) {  
    return n*2;  
}  
int main() {  
    int a = 2;  
    int x = foo(a) + 42;  
}
```

Task 4 (a)

- Consider this Pascal statement:

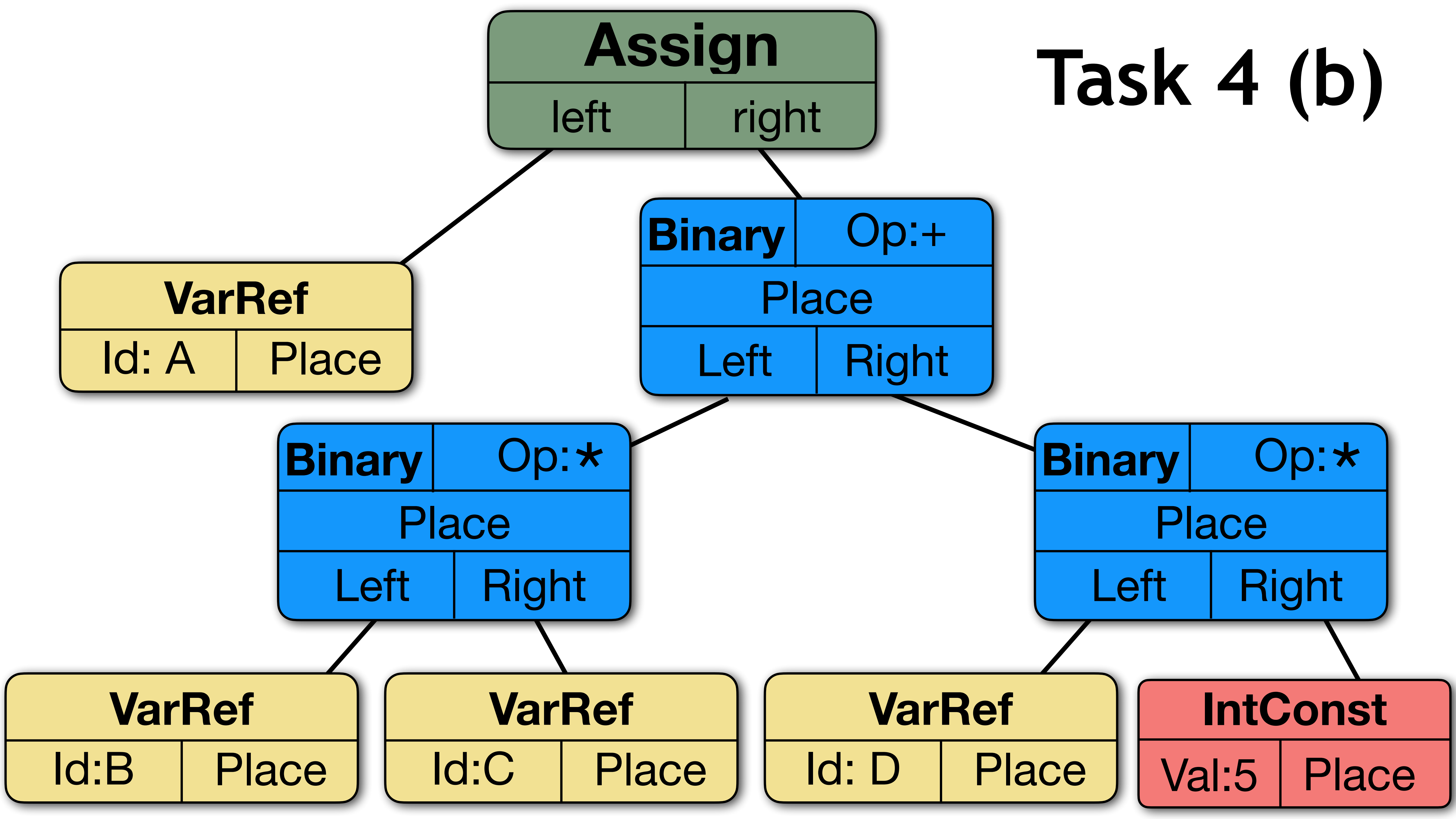


Pascal

```
A := B*C + D*5;
```

- The AST is shown on the next slide.
- Show how the intermediate code generator translates this statement into quadruples, i.e. the values of the `Place` attribute.
- Also show the resulting symbol table.

Task 4 (b)



Task 5

- Translate this program into quadruples:

```
struct S {int a;};  
  
int main() {  
    struct S x;  
    x.a = x.a * 42;  
}
```


Task 6

- Translate this program into quadruples:

```
struct S {int x; int a;};

int main() {
    struct S x[10];
    x[5].a = 42;
}
```

Task 7

- Translate this program into quadruples:

```
struct S {int x; int a;};

int main() {
    struct S *x[10];
    x[5]->a = 42;
}
```

Task 8

- Translate this program into quadruples:

```
struct S {int a;};  
int main() {  
    int i = 0;  
    while (i < 10) {  
        struct S *x[10];  
        x[5]->a = 42;  
        i++;  
    }  
}
```


Task 9

- Consider the following expression:

$$((x * 4) + y) * (y + (4 * x)) + (z * (4 * x))$$

- Show how the value-number algorithm builds a DAG from the expression (remember that + and * are commutative).

Task 10 - Triples (a)

- Triples are similar to quadruples, but save some space.
- Instead of each three-address statement having an explicit result field we let the statement itself represent the result.
- There is no need to insert temporaries into the symbol table.
- The statement $b := (a * 2) + (a * 2)$ would be translated to this triple code:

Nr	Op	Arg₁	Arg₂
(1)	mul	a	2
(2)	mul	a	2
(3)	add	(1)	(2)
(4)	assign	b	(3)

Task 10 - Triples (b)

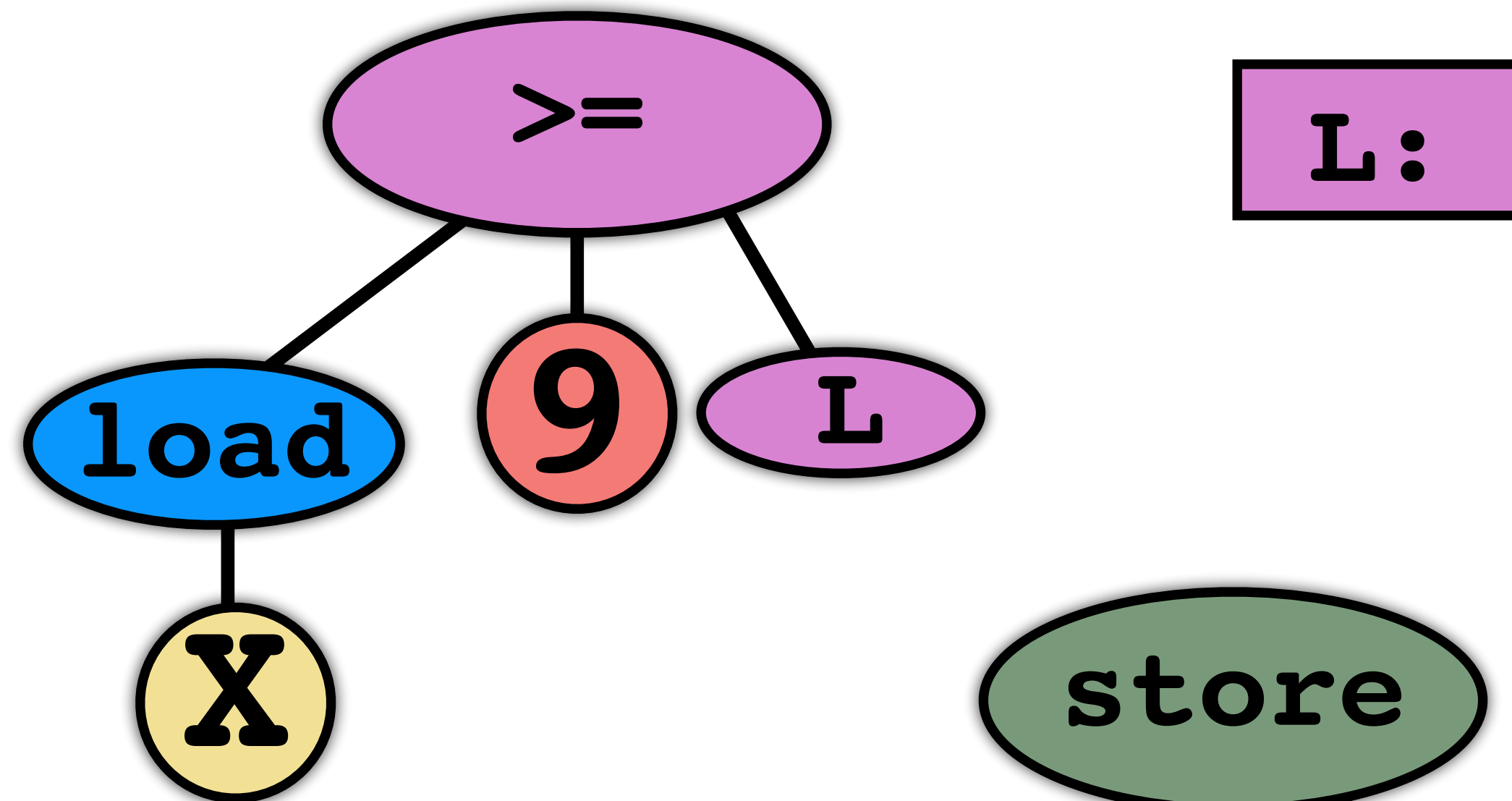
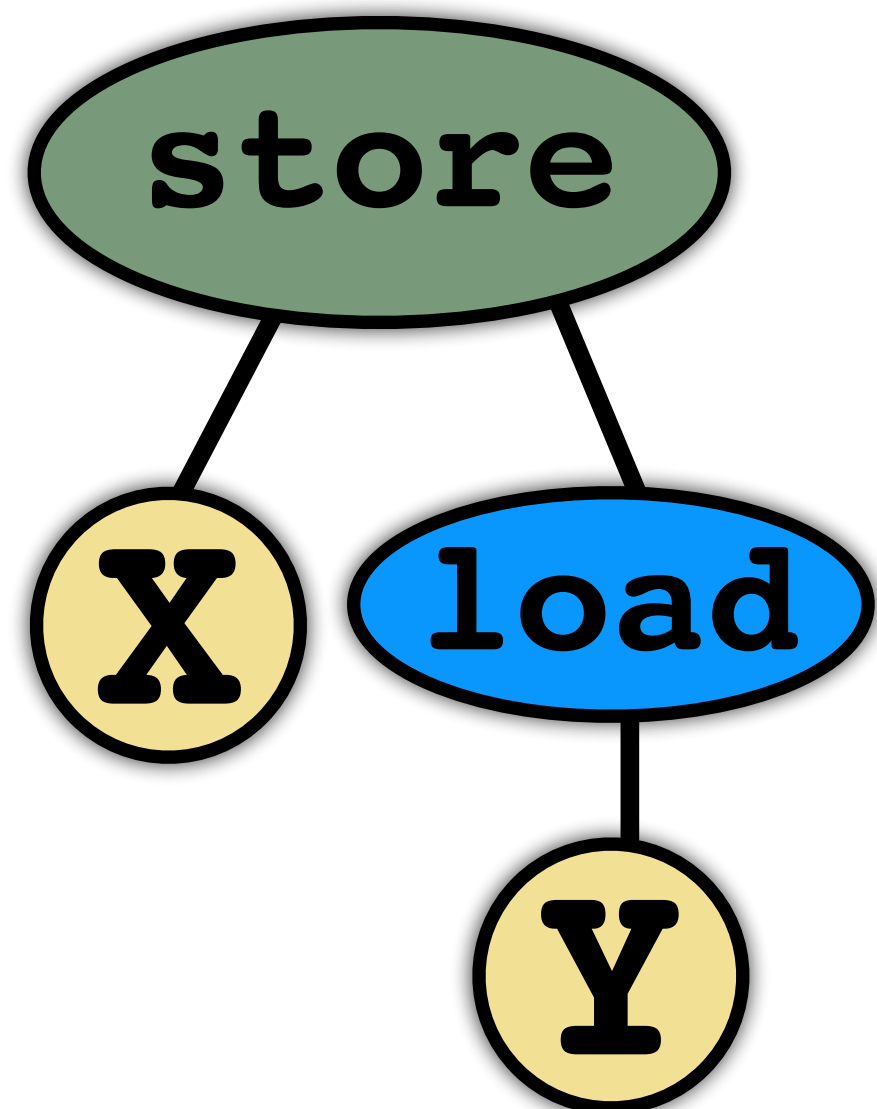
- Translate this program (from Task 2) into triple IR:

```
int main() {  
    int a = 2;  
    int b[5];  
    b[a] = a*3;  
    int x = b[2*a] + 42;  
}
```

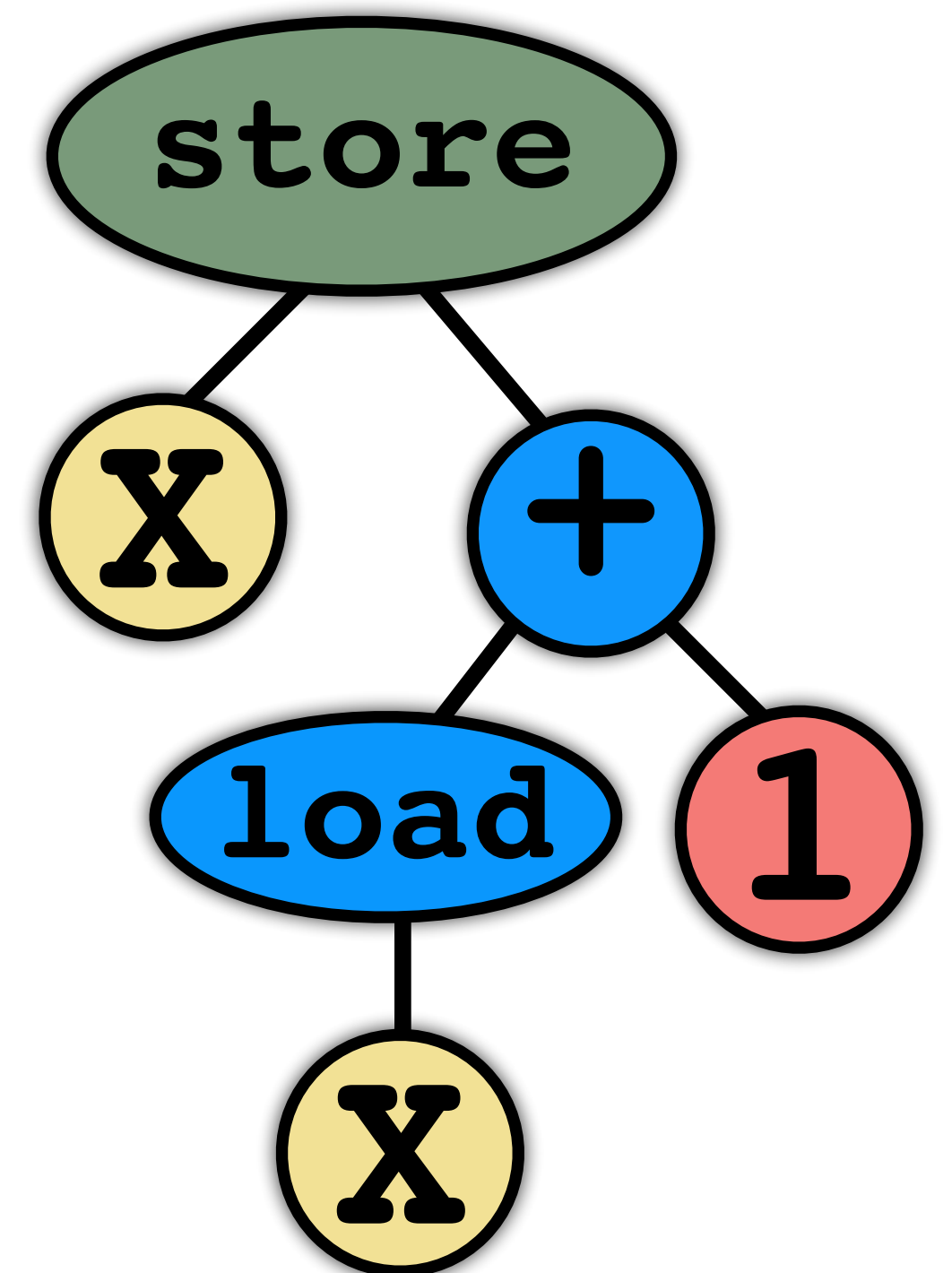
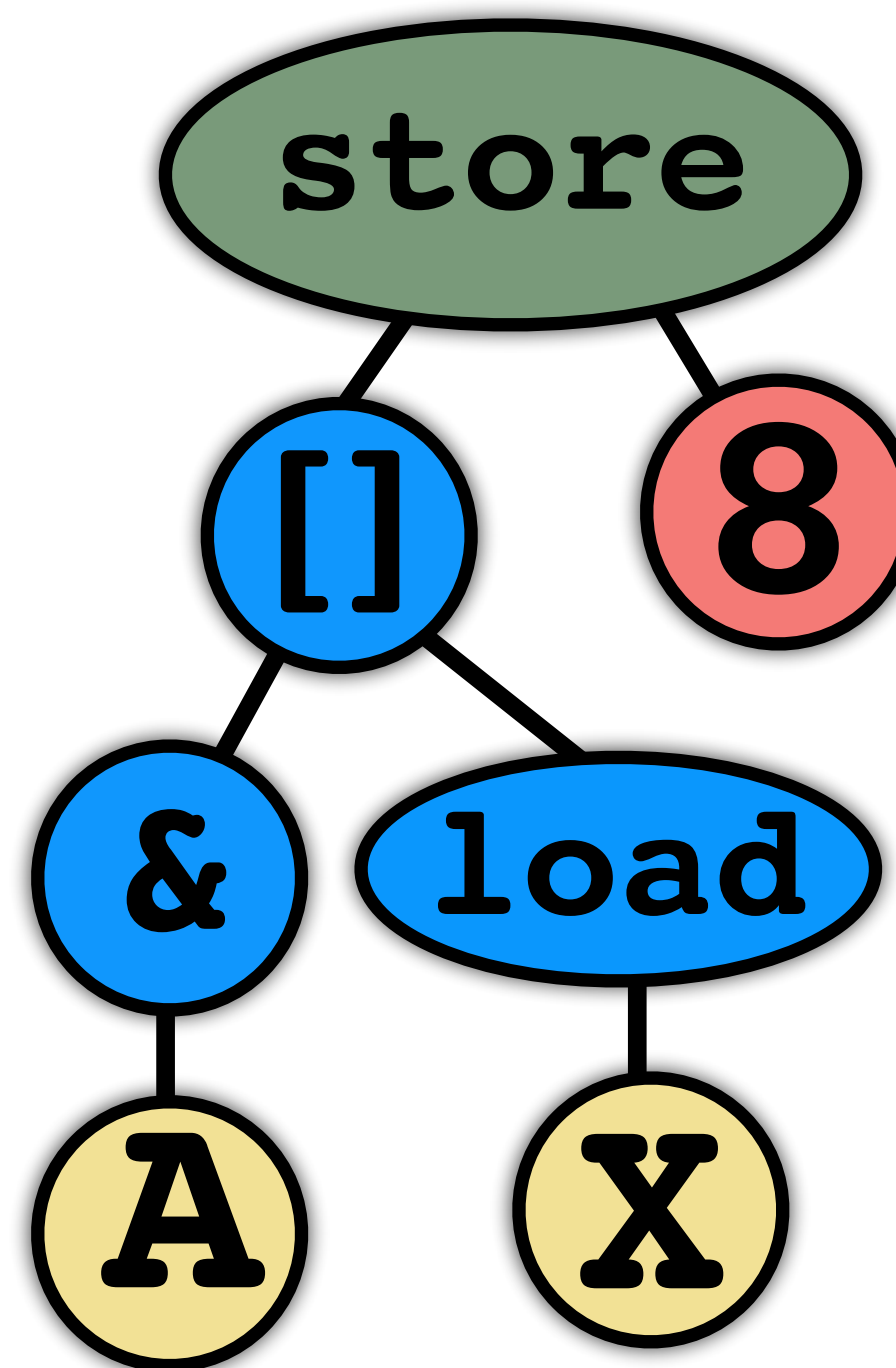



DEFINITIONS AND ALGORITHMS

Sequence of Expression Trees IR



L:



Arithmetic Instructions

Instruction	Semantics
$x \leftarrow y \text{ op } z$	Binary arithmetic or logical operation. Example: Mul, And.
$x \leftarrow \text{op } y$	Unary arithmetic, conversion, or logical operation. Example: Abs, UnaryMinus, Float
$x \leftarrow y$	Copy statement.

Control Flow Instructions

Instruction	Semantics
<code>goto L</code>	Unconditional jump.
<code>if x relop y goto L</code>	Conditional jump. <code>relop</code> is one of <code><</code> , <code>></code> , <code><=</code> , etc. If <code>x relop y</code> evaluates to <code>True</code> , then jump to label <code>L</code> . Otherwise continue with the next tuple. Example: <code>Abs</code> , <code>UnaryMinus</code> , <code>Float</code>
<code>param X;</code> <code>call P, n;</code> <code>V ← returnVal P</code>	1.Make <code>X</code> the next parameter; 2.Make a procedure call to <code>P</code> with <code>n</code> parameters; 3.Assign the return value to <code>V</code>

Data Access Instructions

Instruction	Semantics
$x \leftarrow \text{ADDR}(y)$	Address assignment. Set x to the address of y .
$x \leftarrow y[i]$	Indexed load. Set x to the value in the location i memory units beyond y .
$y[i] \leftarrow x$	Indexed store. Set $y[i]$ to the value of x .
$x \leftarrow \text{IND}(y)$	Indirect load. Set x to the value stored at the address in y .
$\text{IND}(x) \leftarrow y$	Indirect store. Set the memory location pointed to by x to the value held by y .

Quadruples (Quads)

	Result	OP	Arg 1	Arg 2
1	t_1	mul	a	2
2	t_2	mul	a	2
3	t_3	add	t_1	t_2
4		store	b	t_3

Value- Number Algorithm

```
Node InsertNode (  
    Operator op,  
    Node L, R) {  
    V = hashfunc (OP, L, R);  
    N = HashTab.Lookup (V, OP, L, R);  
    if (N == Null) {  
        N = NewNode (OP, L, R);  
        HashTab.Insert (V, N);  
    }  
    return N;  
}
```

COLLBERG.CS.ARIZONA.EDU

LIGERLABS.ORG

SUPPORTED BY
NSF SATC/TTP-1525820
SATC/EDU-2029632

COPYRIGHT © 2024-2026

CHRISTIAN COLLBERG

UNIVERSITY OF ARIZONA